

University of Dhaka

Department of Computer Science and Engineering

Course: CSE-3201 — Operating Systems

Assignment on Deadlock Analysis and Prevention

Submitted By

Name : Farhan Bin Rabbani
Roll No. : 35
Session : 2022–2023

Submitted To

Jargis Ahmed
Lecturer
Department of Computer Science and Engineering
University of Dhaka

Date of Submission

25th July, 2026

*Department of Computer Science and Engineering, University of Dhaka, Dhaka-1000,
Bangladesh*

The scenario states that a centralized computer server has three types of resources (two database connections: DB1 and DB2, two network printers: PR1 and PR2, and a backup storage: BS1). Four applications execute concurrently demanding resources from the centralized server in the following manner:

Process	Currently Allocated	Waiting For
P1	DB1	PR1
P2	PR1	BS1
P3	BS1	DB2
P4	DB2	DB1

1. Resource Allocation Graph:

A Resource Allocation Graph is a directed graph used to model the state of resource assignments and requests in an operating system. It consists of process nodes (circles) and resource nodes (square).

Processes: P1, P2, P3, P4

Resources: DB1, DB2, PR1, PR2, BS1

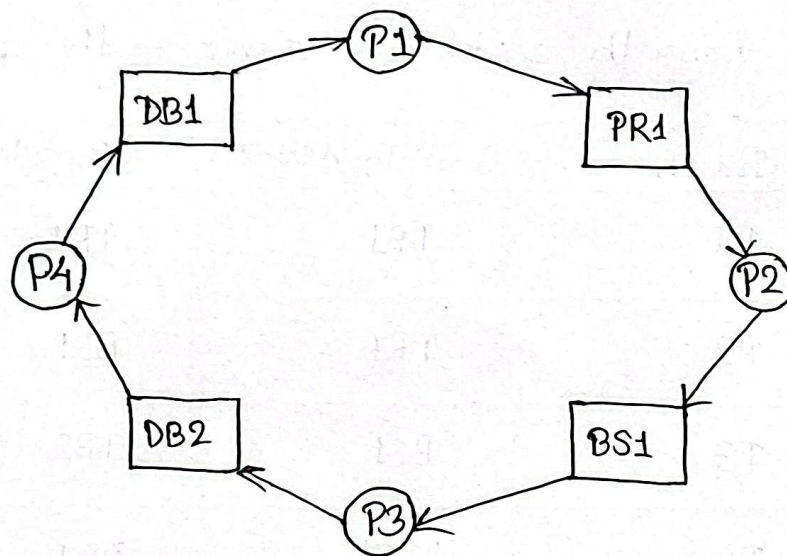
Here, PR2 remains idle as no waiting instance is requesting it.

Allocation Edges (Resource to Process): $DB1 \rightarrow P1$, $PR1 \rightarrow P2$,

$BS1 \rightarrow P3$, $DB2 \rightarrow P4$

Request Edges (Process to Resource): $P1 \rightarrow PR1$, $P2 \rightarrow BS1$,

$P3 \rightarrow DB2$, $P4 \rightarrow DB1$



2. Deadlock Analysis:

Deadlock situation is created amongst a set of processes if each process in the set is waiting for an event that can only be caused by another process in the set. From the resource allocation graph above, it is evident that:

- i) P1 is holding DB1 and waiting for PR1, which can only be released by P2.

- ii) P2 is holding PR1 and waiting for BS1, which can only be released by P3.
- iii) P3 is holding BS1 and waiting for DB2, which can only be released by P4.
- iv) P4 is holding DB2 and waiting for DB1, which can only be released by P1.

Since every process in the cycle is waiting for a resource held by another process in the same cycle, and since no process releases its allocated resource while waiting, the processes face a deadlock situation. The presence of the unallocated resource PR2 doesn't contribute anything as no process waits for it.

3. Explanation of the four necessary conditions:

- i) Mutual Exclusion:- Each resource is either assigned to exactly one process or is available. In the given scenario, the database connections, the printer and the backup storage are all being treated as dedicated non-shareable resources.

ii) Hold-and-Wait: Processes currently holding resources that were granted earlier can request for new ones.

In this scenario, each process holds one resource and waits for another that is held by another process in the same cycle.

iii) No-preemption condition: Resource previously granted can't be forcibly taken away from a process; they have to be explicitly released by the process holding them. Here, no process releases its allocated resource while waiting for another resource.

iv) Circular wait condition: There must be a circular list of two or more processes, each of which is waiting for a resource held by the next member in the chain. From the resource allocation graph, we see that a circular chain of processes and their allocated/requested resources in the order: $P_1 \rightarrow PR_1 \rightarrow P_2 \rightarrow BS_1 \rightarrow P_3 \rightarrow DB_2 \rightarrow P_4 \rightarrow DB_1 \rightarrow P_1$

4. Discussion of prevention/avoidance techniques:

Deadlock prevention strategy lies in building the system in such a manner that one of the four necessary conditions for deadlock situation is never possible:

i) Attacking Mutual Exclusion:- Resources could be made shareable. For the printers (PR1), the company could adopt spooling (processes would "print" to a disk buffer and only a single printer daemon would actually request for physical device. However, resource sharing can't be applied to database connections (DB1, DB2) as they require ~~exclusive~~ locks for data integrity.

ii) Attacking Hold-and-Wait Condition:- Processes could request all necessary resources before any action (if a resource is unavailable, the process is granted nothing and must wait), or they could release all resources before making a new request. However, the processes might end up unnecessarily withholding resources, leading to sub-optimal usage. Furthermore, processes may not know their total requirements in advance.

iii) Attacking No Preemption:- If a process holding a resource is denied a further request, the system can forcibly take away its currently held resource. However this approach can't be done for backups (BS1) as it may result in corrupted data, or for database connections (DB1, DB2) as it may leave them in an inconsistent state.

iv) Attacking Circular-Wait-Condition:- All resources could be assigned a unique number, and the processes could request them any time they like, provided that they follow the strict numerical order of resources. This approach, however, would be difficult in a system with large number of processes and their device requests.

The key to deadlock avoidance is using future resource requests to decide if granting a resource request is safe (a state is safe if the system can guarantee completion of all processes in a certain order, and it is unsafe if there is no guarantee of finishing).

Banker's Algorithm :- When a process requests a resource, the system simulates the allocation and checks if there's at least one sequence of events that ensures every process will finish. If such sequence is found, the request is granted. Otherwise, the request is postponed. Banker's Algorithm fails in situation when the process don't know the maximum amount of resources it requires, or when the number of processes and resource suddenly changes.

5. Comparative Evaluation and Recommendation:

Technique	Strategy	Advantage	Disadvantage
Attack Mutual Exclusion	Make resource shareable	Multiple ^{processes} resources running simultaneously	Inapplicable for resources like database connections
Attack Hold-and-Wait	Request beforehand or release before request	No process is stuck waiting simultaneously	Resources could stay idle, denying faster completion

Technique	Strategy	Advantage	Disadvantage
Attacking no-preemption	Forcibly take held resource	Allow for effective resource utilization	Inapplicable for resources such as database connections and backup storage
Attacking circular wait	Assigning global numerical order	Resource can be ordered requested anytime when order is maintained	Difficult for large number of device requests (perfect for smaller scale)
Avoidance (Banker's Algorithm)	Safety check before granting request	Maximizing resource use by being less restrictive	Inapplicable for changing process or resource counts and where no prior knowledge is available

Recommendation:- Analyzing the privileges and drawbacks of all mentioned strategies, it is recommended for the usage of global numeric ordering (attacking circular wait), as the number of resources don't cause an issue in assigning a satisfiable global order. This is a more practical solution than Banker's Algorithm, especially for difficult to read 'max-need' data.